

9913 Mark Lopes NLP exp 4

An n-gram language model is a type of probabilistic language model that predicts the likelihood of a word based on the previous n-1 words. It is a simple yet powerful model for tasks such as text generation, speech recognition, and other natural language processing tasks.

Key Concepts N-Gram: A sequence of n words. For example:

Unigram: Single word (n=1)

Bigram: Sequence of two words (n=2)

Trigram: Sequence of three words (n=3)

4-gram: Sequence of four words (n=4)

Language Model: A model that assigns probabilities to sequences of words. An ngram language model uses the frequency of n-grams in a training corpus to estimate these probabilities.

Steps to Build an N-Gram Language Model

Tokenization: Split text into words or tokens.

Build N-Grams: Extract n-grams from the text.

Estimate Probabilities: Calculate the probabilities of each n-gram.

Text Generation or Prediction: Use the model to predict the next word in a sequence.

```
In [ ]: import nltk from nltk.corpus import
brown from nltk.tokenize import
word_tokenize nltk.download('brown')
#The Brown Corpus was the first million-word electronic corpus of English, c
# This corpus contains text from 500 sources, and the sources have been cate
```

```
[nltk_data] Downloading package brown to /root/nltk_data...
```

```
[nltk_data] Package brown is already up-to-date!
```

```
Out[ ]: True
```

```
In [ ]: # Loading the corpus

corpus = brown.words()

# Case folding and getting vocab
lower_case_corpus = [w.lower() for w in corpus]

vocab = set(lower_case_corpus) #unique words

##print first 30 tokens
##print first 10 unique words
```

```
In [ ]: print(f"Total words in Corpus: {len(lower_case_corpus)}") #total number of t
print(f"Vocab of the Corpus: {len(vocab)}") #len of vocabulary

Total words in Corpus: 1101192
Vocab of the Corpus: 49815
```

```
In [ ]: #this code is printing unigram, bigram and trigram counts from brown corpus

unigram_counts = {}
bigram_counts = {}
trigram_counts = {}

# Sliding through corpus to get bigram and trigram counts
for i in range(len(lower_case_corpus) - 2):      # Getting
bigram and trigram at each slide      unigram =
lower_case_corpus[i]      bigram = (lower_case_corpus[i],
lower_case_corpus[i+1])
      trigram = (lower_case_corpus[i], lower_case_corpus[i+1], lower_case_corp

      if unigram in unigram_counts.keys():
          unigram_counts[unigram] += 1
      else:
          unigram_counts[unigram]
          = 1

      # Keeping track of the bigram counts
      if bigram in bigram_counts.keys():
          bigram_counts[bigram] += 1
      else:
          bigram_counts[bigram]
          = 1

      # Keeping track of trigram counts
      ##write ur code here      if trigram
      in trigram_counts.keys():
          trigram_counts[trigram] += 1
      else:
          trigram_counts[trigram]
          = 1

      print("Example, count for unigram ('king') is: " + str(unigram_counts['king']
      print("Example, count for bigram ('the', 'king') is: " + str(bigram_counts[(
      print("Example, count for bigram ('the', 'king','of') is: " + str(trigram_co

      """Sample output
      Example, count for unigram ('king') is: 88
      Example, count for bigram ('the', 'king') is: 51
      Example, count for bigram ('the', 'king','of') is: 8"""
```

```
Example, count for unigram ('king') is: 88
Example, count for bigram ('the', 'king') is: 51
      Example, count for bigram ('the', 'king','of') is: 8
Out [ ]: "Sample output\nExample, count for unigram ('king') is: 88\nExample, count for
bigram ('the', 'king') is: 51\nExample, count for bigram ('the', 'kin
g','of') is: 8"
```

N gram language model <https://vitalflux.com/n-gram-language-modelsexplained-examples/>

```
In [ ]: # Function takes sentence as input and suggests possible words that comes af
```

```
def suggest_next_word(input_string, bigram_counts, trigram_counts, vocab):

    # Consider the last bigram of sentence
    tokenized_input = word_tokenize(input_string.lower()) #input gets in low
    last_bigram = tokenized_input[-2:] #get last two words from tokenized in

    # Calculating probability for each word in vocab
    vocab_probabilities = {} #dictionary of word in vocab and its trigram pr
    for vocab_word in vocab:
        test_trigram = (last_bigram[0], last_bigram[1], vocab_word)
        test_bigram = (last_bigram[0], last_bigram[1])

        test_trigram_count = trigram_counts.get(test_trigram,0) #get count o
        test_bigram_count = bigram_counts.get(test_bigram,0) #get denominato

        probability = test_trigram_count / test_bigram_count if test_bigram_
        vocab_probabilities[vocab_word] = probability

    # Sorting the vocab probability in descending order to get top probable
    top_suggestions = sorted(vocab_probabilities.items(), key=lambda x: x[1])
    return top_suggestions
```

```
In [ ]: import nltk
nltk.download('punkt_tab')
suggest_next_word('I am the king', bigram_counts,
trigram_counts, vocab)
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
```

```
Out[ ]: [('james', 0.17647058823529413),
        ('of', 0.1568627450980392),
        ('arthur', 0.11764705882352941)]
```

```
In [ ]: suggest_next_word('I am the king of', bigram_counts, trigram_counts, vocab)
```

```
Out[ ]: [('france', 0.3333333333333333),
        ('hearts', 0.16666666666666666),
        ('kings', 0.08333333333333333)] n
```

gram lang models postlab

Task 1 : Implement a basic n-gram language model (e.g., bigram or trigram) in Python Tokenize a large text corpus into sentences and words. Create a frequency dictionary for n-grams. Calculate the probabilities of each n-gram.

Generate text by predicting the next word based on the previous (n-1) words.

Task 2: Evaluate the model using perplexity. Comment on perplexity measure for bigram as well as trigram

Task 3: Try n gram language modelling using NLTK, SPACY and other libraries.

```
In [ ]: #TAAAAAAAAAAAAAAAAASK 1:
#Implement a basic n-gram language model (e.g., bigram or trigram) in Python
#Create a frequency dictionary for n-grams. Calculate the probabilities of e
#Generate text by predicting the next word based on the previous (n-1) words
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize

corpus = """
The king is wise. The queen is beautiful. The king loves the queen. The
king rules the kingdom with justice and courage.
"""

sentences = sent_tokenize(corpus) # Sentence tokenization
tokenized_sentences = [word_tokenize(sentence.lower()) for sentence in sente

flat_tokens = [word for sentence in tokenized_sentences for word in sentence
```

```
In [ ]: #TAAAAAAAAAAAAAAAAASK 1
from collections import Counter

def build_ngrams(tokens, n=2):
    ngram_counts = Counter()
    for i in range(len(tokens) - n + 1):
        ngram = tuple(tokens[i : i + n])
        ngram_counts[ngram] += 1
    return ngram_counts

bigram_counts = build_ngrams(flat_tokens, n=2)
trigram_counts = build_ngrams(flat_tokens, n=3)
```

```
In [ ]:
```

```

#TAAAAAAAAAAAAAAAAASK 1
c import random

def generate_text(start_words, ngram_prob, n=2, num_words=10):
    output = start_words[:]
    for _ in range(num_words):
        context = tuple(output[-(n-1):])
        candidates = {k[-1]: v for k, v in ngram_prob.items() if k[-1] == context}
        if not candidates:
            break

        next_word = random.choices(list(candidates.keys()), weights=list(candidates.values()))
        output.append(next_word)
    return ' '.join(output)

# Calculate trigram probabilities
trigram_prob = {}
for trigram, count in trigram_counts.items():
    bigram = trigram[:-1]

    bigram_count = bigram_counts.get(bigram, 0)
    if bigram_count > 0:
        trigram_prob[trigram] = count / bigram_count
    else:
        trigram_prob[trigram] = 0

print(generate_text(['the', 'king'], trigram_prob, n=3, num_words=10))

```

the king rules the kingdom with justice and courage .

In []:

```

#TAAAAAASK 2
#Evaluate the model using perplexity. Comment on perplexity measure for bigr
* import math

def perplexity(test_tokens, ngram_prob, n=2):
    N = len(test_tokens)      prob_product = 0
    for i in range(n-1, N):
        ngram = tuple(test_tokens[i-(n-1):i+1])
        # Add smoothing here as well, in case the calculated probability is
    prob = ngram_prob.get(ngram, 1e-9) # small smoothing for unseen ngr
    prob_product *= math.log(prob)      ppl = math.exp(-prob_product / N)
    return ppl test_tokens = word_tokenize("The king rules the kingdom with
courage.").lower

# Calculate bigram probabilities with smoothing bigram_prob
= {}
# Add a small smoothing constant (e.g., 1e-9) to the denominator to avoid di
smoothing = 1e-9 for bigram, count in bigram_counts.items():
    unigram = (bigram[0],)      unigram_count =
unigram_counts.get(unigram, 0)
    bigram_prob[bigram] = (count + smoothing) / (unigram_count + smoothing

# Calculate trigram probabilities (already have smoothing from the get metho
trigram_prob = {} for trigram, count in trigram_counts.items():    bigram =
trigram[:-1]    bigram_count = bigram_counts.get(bigram, 0)
    # Add smoothing here as well, in case the calculated probability is 0
trigram_prob[trigram] = (count + smoothing) / (bigram_count + smoothing

print("Bigram Perplexity:", perplexity(test_tokens, bigram_prob, n=2))
print("Trigram Perplexity:", perplexity(test_tokens, trigram_prob, n=3))

```

Bigram Perplexity: 0.006892550295333958
Trigram Perplexity: 204.00892657095008 In

```

[ ]: #TAAAAAASK 3
# Try n gram language modelling using NLTK, SPACY and other libraries.
from nltk.util import ngrams

bigrams = list(ngrams(flat_tokens, 2))
trigrams = list(ngrams(flat_tokens, 3))

bigram_freq = Counter(bigrams) trigram_freq
= Counter(trigrams)

```

```

In [ ]: import pandas as pd df =pd.DataFrame(list(trigrams),
columns=['w1', 'w2', 'w3']) trigram_counts_df =
df.value_counts()

```

```

In [ ]: import spacy

```

```

nlp = spacy.load("en_core_web_sm") #encoresm contains tokenization, pos etc
doc = nlp(corpus.lower()) tokens = [token.text for token in doc if
token.is_alpha] #convert token to t

bigrams_spacy = list(ngrams(tokens, 2)) #bigram and trigram list using spacy
trigrams_spacy = list(ngrams(tokens , 3))

Counter(bigrams_spacy) #how many times each word occurs
Counter(trigrams_spacy)

```

```

Out[ ]: Counter({'the', 'king', 'is'): 1,
                ('king', 'is', 'wise'): 1,
                ('is', 'wise', 'the'): 1,
                ('wise', 'the', 'queen'): 1,
                ('the', 'queen', 'is'): 1,
                ('queen', 'is', 'beautiful'): 1,
                ('is', 'beautiful', 'the'): 1,
                ('beautiful', 'the', 'king'): 1,
                ('the', 'king', 'loves'): 1,
                ('king', 'loves', 'the'): 1,
                ('loves', 'the', 'queen'): 1,
                ('the', 'queen', 'the'): 1,
                ('queen', 'the', 'king'): 1,
                ('the', 'king', 'rules'): 1,
                ('king', 'rules', 'the'): 1,
                ('rules', 'the', 'kingdom'): 1,
                ('the', 'kingdom', 'with'): 1,
                ('kingdom', 'with', 'justice'): 1,
                ('with', 'justice', 'and'): 1,
                ('justice', 'and', 'courage'): 1})

```

POSTLAB

Q1: What is the role of language models in natural language processing (NLP)? A:

- Predict the next word or sequence of words in a text.
- Measure how likely a sentence or phrase is in a language.
- Support applications like machine translation, speech recognition, and text generation.

Q2: Explain the Markov assumption in the context of language models. A:

- The probability of a word depends only on a fixed number of previous words (not the whole history).
- Simplifies computation in n-gram models.
- Enables efficient estimation of word sequences using limited context.

Q3: How does an n-gram model estimate the probability of a word in a sentence? A:

- Uses frequency counts of n-grams from a corpus.
 - Probability of a word = count of the n-gram / count of the preceding (n-1)gram.
 - Assumes the Markov property to limit context to n-1 words.
-

Q4: What are the limitations of n-gram models for informal/short texts like tweets or SMS? A:

- Cannot handle unseen words or rare n-grams well (data sparsity).
 - Limited context; short texts may not provide enough preceding words.
 - Sensitive to spelling errors, slang, and abbreviations.
-

Q5: Differentiate between static and contextualized word embeddings.

A:

- Static: Each word has a single fixed vector regardless of context (e.g., Word2Vec, GloVe).
 - Contextualized: Vector changes depending on surrounding words (e.g., BERT, ELMo).
 - Contextual embeddings capture word sense and polysemy better.
-

Q6: What is the role of cosine similarity in comparing text vectors? A:

- Measures similarity between two vectors based on direction, not magnitude.
 - Values range from -1 (opposite) to 1 (identical).
 - Commonly used in information retrieval, clustering, and semantic similarity tasks.
-

Q7: What problem does the GloVe model address that Word2Vec does not? A:

- GloVe captures **global word co-occurrence statistics** from the entire corpus.
 - Provides more robust embeddings for rare words.
 - Combines benefits of count-based and predictive approaches.
-

Q8: Describe how FastText represents words using character-level ngrams. A:

- Breaks words into subword n-grams (like 'ing', 'pre', etc.).
 - Word vector = sum of all its subword vectors.
 - Handles rare words and misspellings better than word-level embeddings.
-

Q9: What is data sparsity and how does it affect traditional language models? A:

- Many possible word combinations occur rarely or not at all in a corpus.
 - Leads to zero probabilities for unseen n-grams.
 - Reduces accuracy and generalization of language models.
-

Q10: What is the “curse of dimensionality” in high-dimensional vector spaces? A:

- As dimensions increase, data points become sparse.
 - Distance metrics become less meaningful.
 - Makes learning, clustering, and similarity computation harder.
-

